VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

Dokumentace implementace

SOL26 Interpret

20. dubna 2026

xkapsam00

1 Celkový popis návrhu řešení

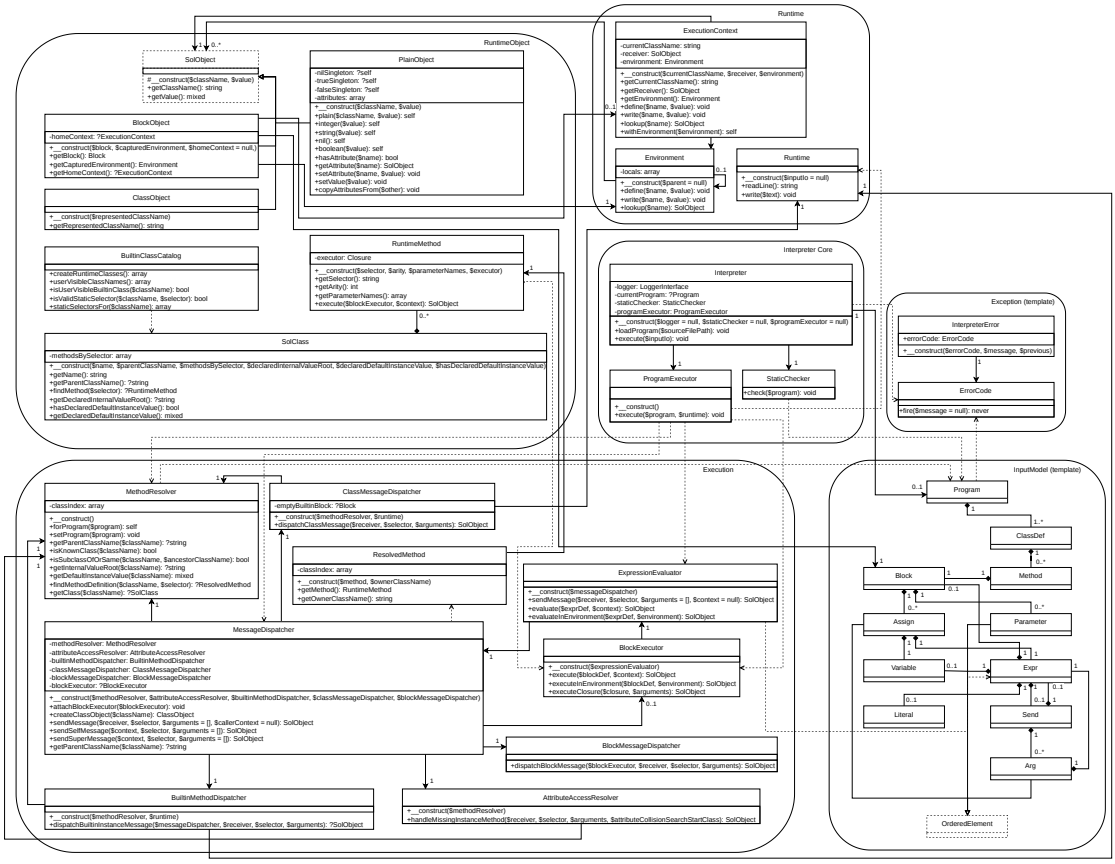
Interpret je rozdělený do několika samostatných vrstev, aby bylo možné oddělit analýzu vstupního programu, statické kontroly, běh programu a reprezentaci runtime objektů. Vstupní třída `Interpreter` nejprve zajistí načtení `Program::fromXml()` a kontrolu programu `StaticChecker::check()`, potom připraví běhové prostředí (`Runtime`) a předá řízení třídě `ProgramExecutor`, která sestaví exekuční komponenty a spustí program.

AST reprezentuje vnitřní strukturu programu po načtení vstupu. AST z XML je reprezentované třídami `InputModel`; stejné struktury pak používá statická kontrola (`StaticChecker`) i běh interpretu.

Statická kontrola `StaticChecker::check()` se spouští nad sestaveným `AST` před samotným `runtime` a zachytává chyby co nejdříve. Kontroluje např. existenci vstupního bodu `Main»run`, duplicity tříd a metod, shodu arity selektorů s parametry bloku, validitu dědičnosti a použití nedefinovaných identifikátorů.

Samotná interpretace je soustředěna do vrstvy `Runtime` a `Execution`. `ProgramExecutor` inicializuje kontext hlavního spuštění, tj. při startu běhu nejprve vytvoří `MethodResolver` nad aktuálním `Program`, následně centrální `MessageDispatcher`, potom `ExpressionEvaluator` a `BlockExecutor`; následně se nastaví jejich spolupráce tak, že výrazy se vyhodnocují přes evaluator, který pro posílání zpráv používá `MessageDispatcher`, a kdykoli je potřeba vykonat blok, dispatcher předá řízení `BlockExecutoru`. Po této inicializaci se spustí vstup programu vytvořením instance třídy `Main` a voláním metody `run`.

Diagram zahrnuje i třídy dodané šablonou (část `InputModel` a `Exception`), které jsou v UML odděleny rámečky s popisem `InputModel (template)` a `Exception (template)`.



Obrázek 1: UML diagram architektury interpretu

2 Hlavní interní datové struktury

- **`Program`, `ClassDef`, `Method`, `Block`, `Expr`, ...**
  Tyto třídy tvoří `AST` načteného programu, tedy jeho interní strukturní reprezentaci před spuštěním. Nad stejným modelem pak probíhá jak statická kontrola, tak následná interpretace.

- **`Environment`**
  Reprezentuje mapování identifikátorů (tj. názvů proměnných a parametrů) na runtime objekty v aktuálním scope. Tato struktura se používá při čtení a zápisu proměnných během vyhodnocování výrazů, při navázání argumentů metod/bloků na parametry a také při zachycení lexikálního okolí u bloků.

- **`ExecutionContext`**
  Nese aktuální kontext provádění (např. `receiver`, aktivní prostředí `Environment`, případně metadata potřebná při voláních). Umožňuje předávat stav mezi evaluátorem výrazů, dispatchery a vykonávačem bloků bez globálních proměnných.

- **`SolObject` a jeho potomci (`PlainObject`, `ClassObject`, `BlockObject`)**
  Objekty jsou rozdělené na `PlainObject`, `ClassObject` a `BlockObject`, protože každý typ nese jiný druh runtime chování (např. `ClassObject` se používá pro vytvoření instance přes `new`). Společný předek `SolObject` pak umožňuje, aby je interpret mohl předávat a zpracovávat jednotně bez ohledu na konkrétní typ (např. uložit je do proměnné v `Environment`).

- **`RuntimeMethod` a `ResolvedMethod`**
  `RuntimeMethod` reprezentuje metodu připravenou ke spuštění, zatímco `ResolvedMethod` je výsledek vyhledání metody v konkrétním kontextu. Toto oddělení umožňuje mít zvlášť fázi vyhledání a zvlášť fázi vykonání.

- **`SolClass` a `BuiltinClassCatalog`**
  `SolClass` reprezentuje runtime metadata třídy, zejména vazbu na rodičovskou třídu, mapu metod a výchozí hodnotu interní reprezentace instance. Tyto informace jsou inicializovány přes `BuiltinClassCatalog`, který centralizuje definice vestavěných tříd a poskytuje jejich jednotný popis pro další části interpretu.

3 Návrhové vzory a principy OOP

- **Princip jedné odpovědnosti a vrstvení**
  Kód je rozdělený do vrstev `InputModel`, `Analysis`, `Execution`, `Runtime` a `RuntimeObject`, kde každá vrstva řeší jiný problém (parsování modelu, statickou kontrolu, běh programu, reprezentaci runtime hodnot). Tento princip je v projektu použit hlavně proto, aby se změny držely lokálně: úprava statických kontrol typicky nezasahuje do exekuce a naopak. Z pohledu organizace to snižuje provázanost mezi třídami a z pohledu rozšiřitelnosti to umožňuje přidávat nové kontroly nebo runtime chování bez zásahů napříč celou architekturou.

- **Delegace (Dispatcher/Strategy) pro posílání zpráv**
  Posílání zpráv je centralizované v `MessageDispatcher`, který deleguje konkrétní případy na specializované komponenty (`BuiltinMethodDispatcher`, `ClassMessageDispatcher`, `BlockMessageDispatcher`, případně `AttributeAccessResolver`). Důvodem je oddělit rozhodování podle typu receiveru od samotné evaluace výrazů a zabránit jedné přetížené třídě s rozsáhlou podmínkovou logikou. Přínos je hlavně v rozšiřitelnosti: nové pravidlo dispatchingu lze přidat jako izolovanou změnu v jedné komponentě místo úprav velkého `switch`/`if`-řetězce.

- **Polymorfismus runtime objektů**
  Runtime hodnoty sdílejí společný základ `SolObject`, zatímco konkrétní chování je rozdělené do `PlainObject`, `ClassObject` a `BlockObject`. Tento OOP přístup je použit proto, aby interpret pracoval s jednotným typem hodnot a nemusel na více místech ručně rozlišovat všechny konkrétní varianty. V organizaci kódu to vede k čistším rozhraním mezi komponentami a z hlediska rozšíření usnadňuje přidání nového druhu runtime objektu bez přepisování celé exekuční pipeline.
- **Factory přístup při vytváření runtime objektů**
  Vytváření objektů je soustředěné do dedikovaných míst (např. `ClassMessageDispatcher` pro `new`/`from:`, statické tovární metody v `PlainObject`, katalog vestavěných tříd `BuiltinClassCatalog`). Důvodem je mít jednotná pravidla inicializace objektů, včetně výchozích hodnot a validace kompatibility u `from:`. Přínos je v tom, že změna konstrukce objektů proběhne na jednom místě a nevyžaduje zásahy do všech částí, které objekty používají.
- **Singleton pro základní konstantní hodnoty**
  U objektů `Nil`, `True` a `False` je v `PlainObject` použitý přístup singletonu, tj. v runtime existuje vždy jedna sdílená instance dané hodnoty. Důvodem je sjednocení identity těchto konstant a jednodušší porovnávání/reference v běhu interpretu. Z hlediska organizace jde o centralizované místo tvorby těchto objektů a z hlediska rozšíření je zachované jednotné chování těchto konstant napříč celou exekucí.

4 Největší problémy a jejich řešení

- **Lookup metod a fallback na instanční atribut**
  Sporná situace byla přesně definovat pořadí řešení zprávy v případech, kdy může existovat metoda i stejnojmenný atribut. Bez pevného pořadí vznikaly nejednoznačné výsledky a chování se lišilo podle kontextu volání. Řešení bylo zavést jednotné pravidlo v dispatchingu: nejdřív lookup metody přes dědičnost, až potom fallback na atribut; u setteru se navíc kontroluje kolize s metodou a v takovém případě se vrací `INT_INST_ATTR` (54).
- **Runtime chování `timesRepeat:` pro neblokové argumenty**
  U built-in metody `timesRepeat:` bylo potřeba přesně ošetřit případ, kdy argument není blok (nebo obecně objekt, který nerozumí `value:`). Bez této kontroly by interpret padal v pozdější fázi méně srozumitelnou chybou. Řešení je explicitní runtime guard v `BuiltinMethodDispatcher`, který ověřuje podporu `value:` a při nesplnění vrací doménovou chybu `INT_DNU` (51).
- **Dědičnost tříd a validace class hierarchy**
  Prakticky problematické bylo sjednotit statická pravidla dědičnosti s runtime lookupem metod. Hraniční stavy byly hlavně neexistující rodičovská třída a cyklická dědičnost, které mohou rozbít lookup i inicializaci runtime tříd. Řešení kombinuje statické ověření v `StaticChecker` (existence parenta, detekce cyklu) a následně jednotný runtime lookup přes `MethodResolver`.

5 Zásahy do šablony

- `int/src/interpreter/InputModel/Block.php` – doplněna statická metoda `Block::empty()`, která vytváří prázdný built-in blok (arita 0, bez parametrů a bez příkazů). Tato úprava byla potřeba pro korektní runtime konstrukci `Block new`/`Block from:` bez ručního skládání AST uzlu na více místech.

6 Rozsah a důvod úprav ve vylepšené verzi

- Byl upraven běh uzávěrů v runtime tak, aby přiřazení ve vnořených blocích zapisovalo do nejbližší existující lexikální vazby. Změna je realizována přes `write()` v `Environment`/`ExecutionContext` a její

použití v `BlockExecutor`; díky tomu se změny proměnných správně propisují napříč closure scope.

- Byla zpřesněna pravidla dispatchingu vestavěného chování. Metoda `timesRepeat:` nyní přijímá argument podle toho, zda rozumí zprávě `value:` (ne pouze podle typu `Block`), byla doplněna korektní class-side konstrukce `Block new`/`Block from:` a sjednoceno singleton chování `Nil`/`True`/`False` v `PlainObject`.

7 Možnosti dalšího rozšiřování

7.1 Rozšíření 1 – Mechanismus `doesNotUnderstand`

Toto rozšíření dobře navazuje na současný návrh, protože neúspěšný lookup zprávy je už nyní centralizovaný v dispatchi (`MessageDispatcher` + `AttributeAccessResolver`) a končí chybou `INT_DNU`. Implementace by spočívala v tom, že místo okamžité chyby by se v tomto bodě vytvořil objekt zprávy (selektor + argumenty) a receiveru by se poslala zpráva `doesNotUnderstand:`; výchozí implementace v `Object` by zachovala současné chování (chyba 51), zatímco uživatelské třídy by ji mohly přepsat. Díky lokalizaci této logiky do jedné části dispatchingu by šlo změnu provést bez zásahu do evaluace výrazů nebo správy scope.

7.2 Rozšíření 2 – Podpora rozhraní (`interface`)

Toto rozšíření je vhodné hlavně proto, že většina práce leží ve statické fázi, kterou už má interpret oddělenou v `StaticChecker`. Rozšíření by vyžadovalo doplnit reprezentaci rozhraní do `InputModel` a přidat statické kontroly, že třídy deklarující rozhraní skutečně implementují požadované selektory (přímo nebo zděděně), včetně kontroly dědičnosti rozhraní. Runtime exekuce by se měnila minimálně, protože samotné volání metod může zůstat beze změny; Rozhraní by v tomto návrhu fungovala hlavně jako formální kontrola povinných metod, která se vyhodnotí už ve statické fázi před spuštěním.

8 Využití AI nástrojů

V projektu jsem využil nástroje generativní AI (ChatGPT) jako konzultační podporu při vývoji interpretu. AI mi sloužila především k dovysvětlení některých aspektů jazyka PHP, například principů implementace tříd a jejich vzájemné komunikace. Dále jsem ji využíval ke konzultaci návrhu řešení, zejména pro získání různých pohledů na konkrétní problém a ověření, zda zvolený postup dává smysl. Pomáhala mi také s vysvětlením použití vybraných funkcí a metod, s hledáním konkrétních chyb v implementaci a také při tvorbě `smoke` testovací sady.

Typický pracovní postup spočíval v tom, že jsem nejprve navrhl vlastní základní řešení problému a následně jej konzultoval s AI. Na základě těchto konzultací jsem identifikoval chyby v implementaci, případně získal návrhy na úpravy a doplnění. AI byla využita pro psaní komentářů v kódu a pro stylistickou úpravu kódu i dokumentace.